CampusFlow Inc.

Campus Flow

Software Requirements Specification

Version <1.0>

Revision History

Date	Version	Description	Author
22-March 2025	1.0	Initial Draft based on project requirements	Khapil Ibsan Hossain Tausif Tusar Arif Hossain

Table of Contents

1. **Introduction**
   - 1.1 Purpose
   - 1.2 Scope
   - 1.3 Definitions, Acronyms, and Abbreviations
   - 1.4 References
   - 1.5 Overview
2. **Overall Description**
   - 2.1 Use-Case Model Survey
   - 2.2 Assumptions and Dependencies
3. **Specific Requirements**
   - 3.1 Use-Case Reports
   - 3.2 Supplementary Requirements
4. **Supporting Information**

1.Introduction

1.1 Purpose

The document is created to present the specifications of the AI-enabled online college program management system. This document aims to explain the functionality of the system and how users interact with it. Additionally, it presents the AI features and the general structure of the project. This is to ensure that all users understand how the system works.

1.2 Scope

This system would enable users to manage a college program in one place. The features would include course registration, grading, reviews, complaints, and graduation. The users of this system would be registrars, teachers, students, and visitors. Additionally, this system would have an AI assistant to help users with their questions and provide information on the courses and the system.

1.3 Definition, Acronyms and Abbreviations

AI: Artificial Intelligence for assisting the user through a chatbot

LLM: Large Language Model for assisting the user when local system data is not sufficient

GPA: Grade Point Average for evaluating student performance

GUI: Graphical User Interface for user interaction with the system

Registrar: Admin user for managing the system

Instructor: User for teaching the course and assigning grades

Student: User for enrolling in the course

Visitor: User for browsing and applying

1.4 References

Firebase Documentation: https://firebase.google.com/docs

React.js Documentation: https://react.dev/

Express.js Documentation: https://expressjs.com/

Cunyfirst: https://www.cuny.edu/about/administration/offices/cis/cunyfirst/

1.5 Overview

The rest of this document is divided into different sections. The description of the overall system explains the system and its users. The description of specific requirements explains the system's features and functionality. The description of supporting information includes further information and diagrams regarding the system.

2. Overall Description

The AI-enabled online college program management system is a comprehensive web-based system designed and created to manage college programs in a centralized manner. The system has been designed to support four different user role categories. These four categories of users in the system are visitors, students, instructors, and registrars. The system has been equipped with several functional capabilities, which include course registration, grade management, application processing, course evaluation, grievance handling, and graduation verification. The system has also been integrated with an AI chatbot assistant, which can be accessed by users at all four roles. The AI chatbot assistant has been integrated to address system-related queries and academic-related queries by querying its knowledge base or using a connected external large language model. The system has been created using React.js, Express.js, and Firebase. The system has been created to provide a secure, accurate, and accessible GUI.

2.1 Use-Case Model Survey

The AI enabled online college program management system is a GUI-based system that has been developed to manage all aspects of a college program within a unified interface. The system has been developed to support four different user roles. These are registrars, instructors, students, and visitors. The system has been developed to support various features that are part of a complete academic lifecycle. These features include student application, instructor application, semester period, course registration, class reviews, grading, complaint handling, graduation, etc. The system has been developed to support a chatbot that uses a local vector database for handling queries. The chatbot has been developed to use a large external language model. The system has been developed to be consistent, self-contained, and navigable within a single GUI window.

- Visitors: Operating as unauthenticated users, Visitors are granted read-only access to public-facing system metrics displayed on the landing page, including comprehensive course catalogs, aggregate course ratings and top-tier student GPA statistics. To gain authenticated access, Visitors may submit formal applications for either Student or Instructor roles, both of which are strictly subject to manual review and approval by a Registrar.
- Students: Serving as the primary academic end-users, Students are authenticated entities provisioned with unique identification credentials upon admission. During designated registration windows, Students are authorized to enroll in two to four courses per

semester, contingent upon the absence of scheduling conflicts and available class capacity. Post-enrollment capabilities include submitting quantitative (star ratings) and qualitative course evaluations, filing official grievances against peers or faculty, and initiating graduation applications upon the successful completion of the requisite eight courses. Furthermore, Student accounts are governed by automated academic standing protocols, which enforce warnings, academic suspension, or system termination based on predefined GPA thresholds and behavioral compliance.

- Instructors: Functioning as the academic faculty, Instructors are authenticated users officially assigned to course modules by Registrars. Their system access is strictly scoped to view the basic profiles and academic records of Students currently enrolled in their assigned rosters. Primary operational responsibilities include the submission of final grades during defined grading periods and the management of course waitlists. Instructor accounts are monitored by system compliance protocols and are subject to progressive disciplinary actions ranging from formal warnings to account suspension or termination triggered by excessive course cancellations, grading anomalies, consistently poor course evaluations, or unresolved administrative complaints.
- Registrars: Operating as the system administrators, registrars possess unrestricted global access and oversight across the platform. Their core operational mandate involves managing the four distinct semester lifecycle phases: class configuration, course registration, active term execution, and grading. Administrative responsibilities encompass the adjudication of all incoming Student and Instructor applications, course catalog generation, faculty assignment, capacity configuration, grievance resolution, and the final verification of graduation applications. Additionally, Registrars maintain the global system dictionary of prohibited terms utilized by the automated course review filter and retain comprehensive visibility into all system audit logs and user activities.
- AI Chatbot Assistant: Integrated via a dedicated, persistent interface component, the AI chatbot provides role-based conversational assistance to all user tiers. Query scopes are dynamically restricted based on authentication status: Visitors are limited to general program and course inquiries; Students may query data specific to their active enrollments; and Instructors may request insights pertaining to their assigned class rosters. Architecturally, the chatbot processes inputs by first querying an internal vector database populated with institution-specific data. If the query exceeds the local knowledge base, the system delegates the request to an external Large Language Model , appending a mandatory system disclaimer regarding the potential for AI hallucinations in the generated response.

2.2 Assumptions and Dependencies

Assumptions

- System Scale & Operational Environment: The system is explicitly designed as a proof-of-concept academic environment for a simulated institution. It is assumed that the system architecture is not required to scale beyond a concurrent user base of approximately ten students to adequately demonstrate core functional capabilities.
- User Interface Navigation: All user interactions are assumed to occur within a single, cohesive Graphical User Interface window, strictly avoiding the use of external pop-up windows or fragmented browser navigation states.
- Data Accuracy & Administrative Maintenance: It is assumed that end-users will supply accurate information during account registration and application workflows. Furthermore, the integrity of institutional data and the active management of academic timelines rely entirely on consistent maintenance by the Registrar role.
- Role-Based Access & Automation: The system assumes strict, mutually exclusive role assignments executed accurately upon account provisioning. Additionally, business logic governing academic standing including GPA-based admissions, probationary warnings, suspensions, and terminations which is assumed to be enforced automatically by the system, requiring Registrar intervention exclusively for edge cases demanding manual adjudication.
- Temporal State Management: Workflows dependent on academic calendars, such as course registration and final grading, assume that semester lifecycle phases are explicitly configured and transitioned accurately by the Registrar.

Dependencies

- Core Technology Stack: The platform's architectural integrity and continuous operation are fundamentally dependent on React.js for client-side GUI rendering, Express.js for server-side API routing, and Firebase for backend NoSQL data persistence and user authentication.
- Hybrid AI Architecture: The chatbot's functionality relies on a dual-dependency model. It is dependent on a local vector database for the storage and retrieval of institution-specific data. For generalized queries exceeding the local knowledge scope, the system is strictly dependent on an external Large Language Model API.
- Degraded State Operation: It is recognized that any network disruption or service outage affecting the external LLM API will result in a degraded state of chatbot functionality. Under such conditions, the chatbot's operational capacity will be limited exclusively to resolving queries capable of being serviced by the functional local vector database.

3. Specific Requirements

The AI-enabled college management system identifies the key functionalities of the system, which are required for its operation. The system must have different user roles, including registrars, instructors, students, and visitors, as well as different functionalities like user registration, course management, grading, and complaints. The system will have an AI-powered assistant that will help answer the queries of the users. The system must also have security, accuracy, and usability, as well as adhere to the system rules.

3.1 Use-Case Reports

Use Case: Browse System Information

- **Actor(s):** Visitor, Student
- **Description:** Users can browse general system information such as courses, top students, and program details.
- **Preconditions:** User is on home page
- **Postconditions:** Information is displayed correctly
- **Functional Requirements:**
    - FR1 – Display courses and descriptions
    - FR2 – Display top GPA students and statistics

Use Case: Register / Login

- **Actor(s):** Visitor, Student, Instructor, Registrar
- **Description:** Users register and or log into the system.
- **Preconditions:** None (register) or valid account exists (login)
- **Postconditions:** User is authenticated and redirected to dashboard
- **Functional Requirements:**
    - FR3 – Create user account
    - FR4 – Validate login credentials
    - FR5 – Assign user roles

Use Case: Apply as Student

- **Actor(s):** Visitor, Registrar
- **Description:** Visitor applies to become a student and application is approved or rejected by the registrar.
- **Preconditions:** Visitor fills out application
- **Postconditions:** Student account is created or rejected
- **Functional Requirements:**

    - FR6 – Submit application
    - FR7 – Evaluate student's GPA and capacity
    - FR8 – Approve/reject application

Use Case: Register for Courses

- **Actor(s):** Student
- **Description:** Students select and register for courses.
- **Preconditions:** Student logged in, registration period active
- **Postconditions:** Student is enrolled in courses
- **Functional Requirements:**
    - FR9 – Display available courses
    - FR10 – Check time conflicts
    - FR11 – Handle waitlist

Use Case: Manage Courses

- **Actor(s):** Registrar
- **Description:** Registrar manages courses.
- **Preconditions:** Registrar logged in
- **Postconditions:** Courses updated in system
- **Functional Requirements:**
    - FR12 – Create courses
    - FR13 – Assign instructors
    - FR14 – Set class size

Use Case: Assign Grades

- **Actor(s):** Instructor
- **Description:** Instructor assigns grades to students.
- **Preconditions:** Grading period active
- **Postconditions:** Grades stored in system
- **Functional Requirements:**
    - FR15 – Input grades
    - FR16 – Store academic records

Use Case: Submit Reviews

- **Actor(s):** Student
- **Description:** Students rate courses and submit reviews.
- **Preconditions:** Student enrolled in course

- **Postconditions:** Review stored or flagged
- **Functional Requirements:**
    - FR17 – Submit rating
    - FR18 – Filter inappropriate content

Use Case: File Complaint

- **Actor(s):** Student, Instructor, Registrar
- **Description:** Users submit complaints which registrar reviews.
- **Preconditions:** User logged in
- **Postconditions:** Complaint resolved
- **Functional Requirements:**
    - FR19 – Submit complaint
    - FR20 – Process complaint
    - FR21 – Issue warnings

Use Case: AI Chatbot Interaction

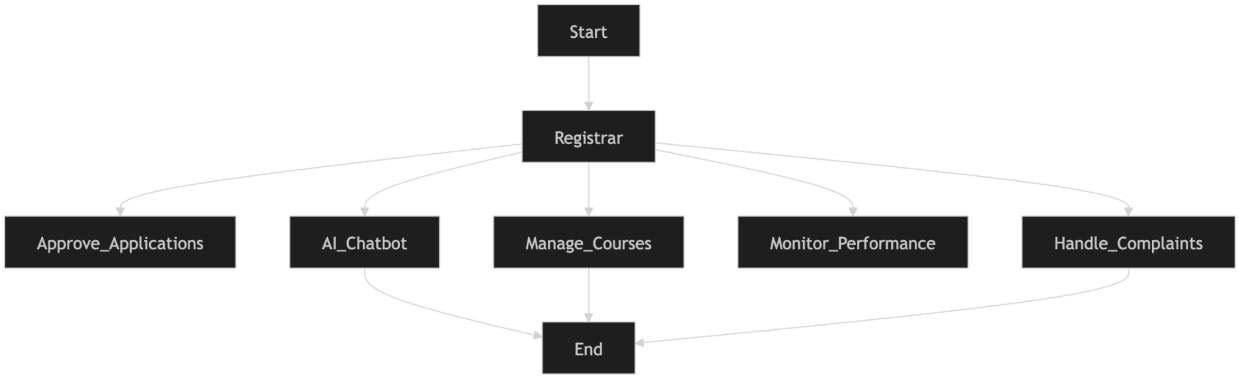
- **Actor(s):** All Users
- **Description:** Users interact with AI assistants for help and recommendations.
- **Preconditions:** Chat interface available
- **Postconditions:** Response displayed
- **Functional Requirements:**
    - FR22 – Process user queries
    - FR23 – Retrieve from knowledge base
    - FR24 – Query external LLM

Use Case: Apply for Graduation

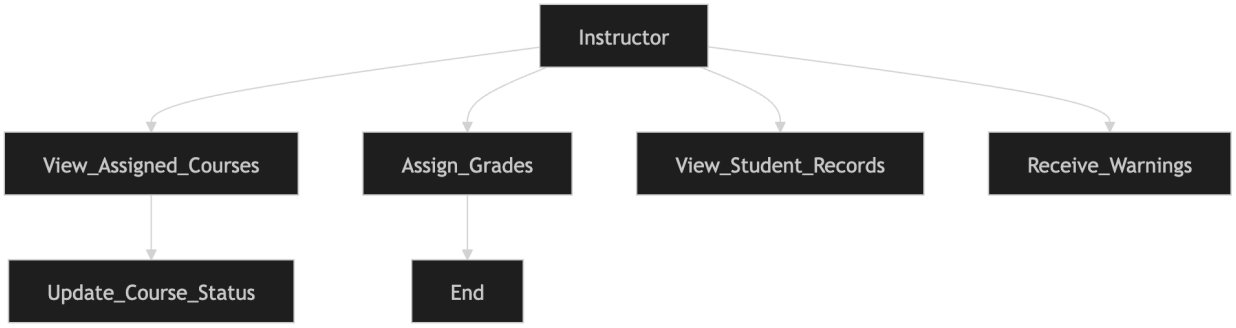
- **Actor(s):** Student, Registrar
- **Description:** Student applies for graduation and registrar verifies requirements.
- **Preconditions:** Student completed courses
- **Postconditions:** Graduation approved or rejected
- **Functional Requirements:**
    - FR25 – Submit graduation request
    - FR26 – Verify requirements

3.2 Supplementary Requirements

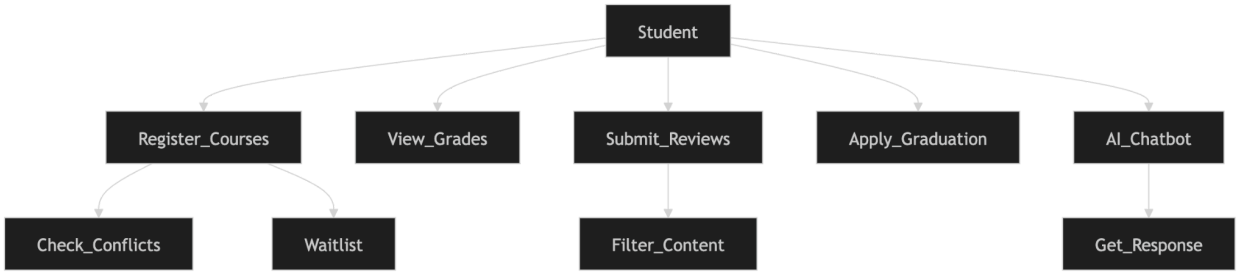
3.2.1 Registrar Use Case Model



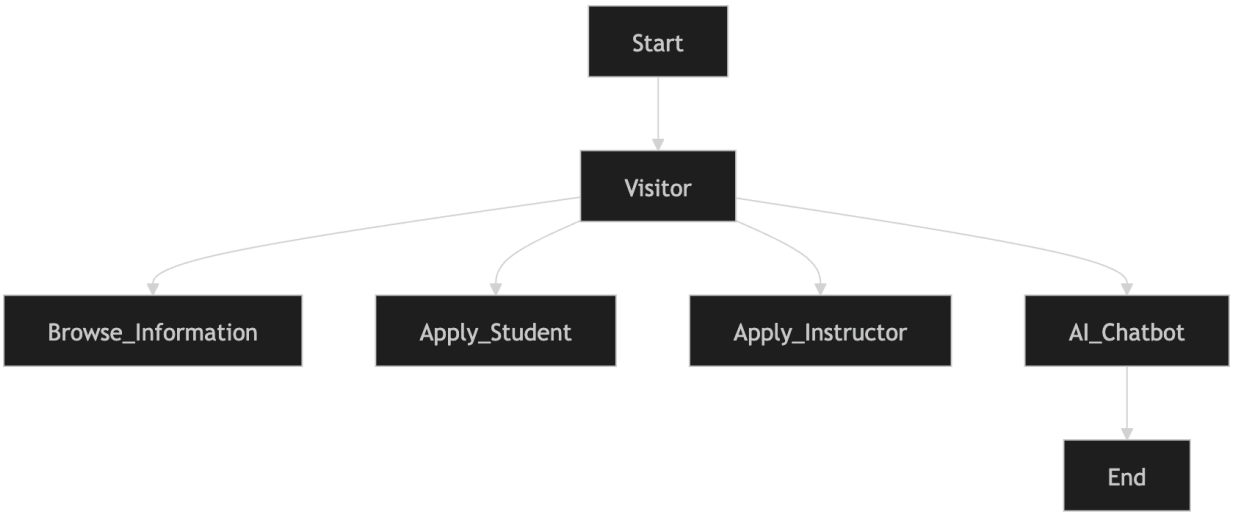
3.2.2 Instructor Use Case Model



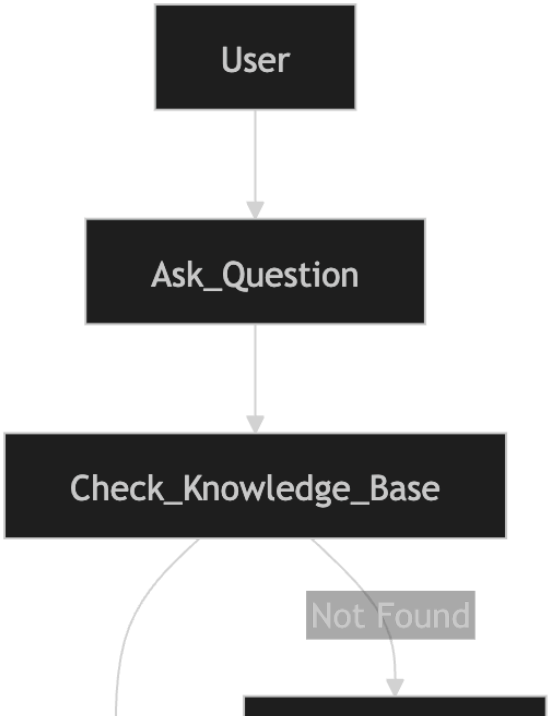
3.2.3 Student Use Case Model



3.2.4 Visitor Use Case Model



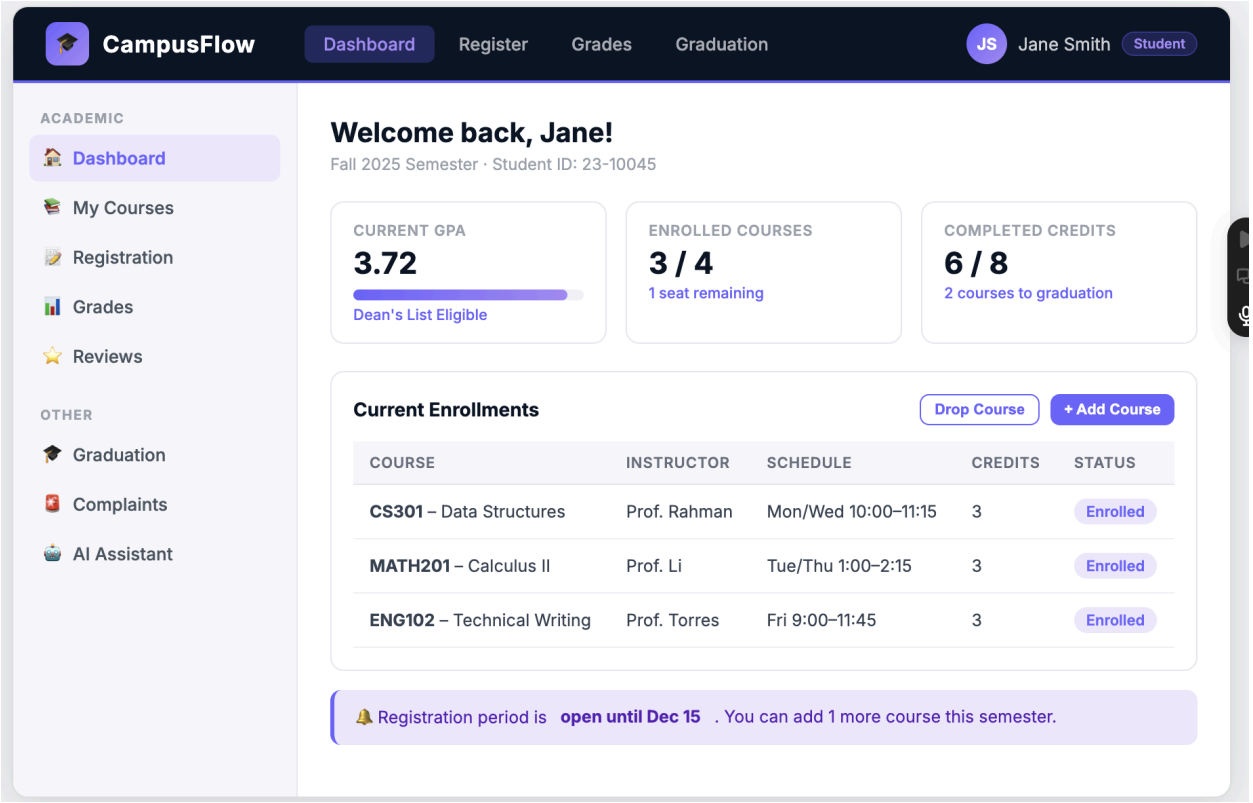
3.2.5 AI Chatbot Use Case Model



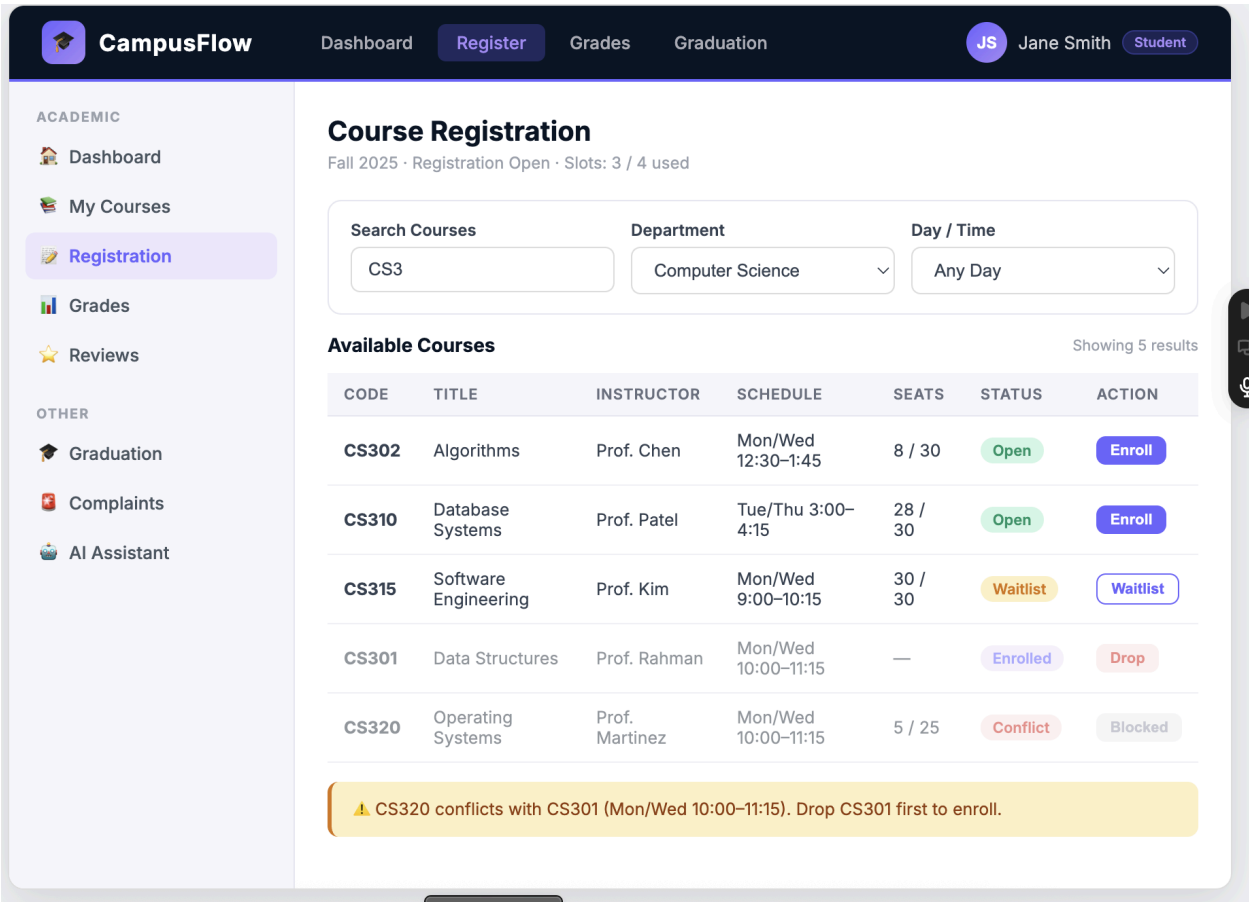
4. Supporting Information

Images below show a mockup of our application

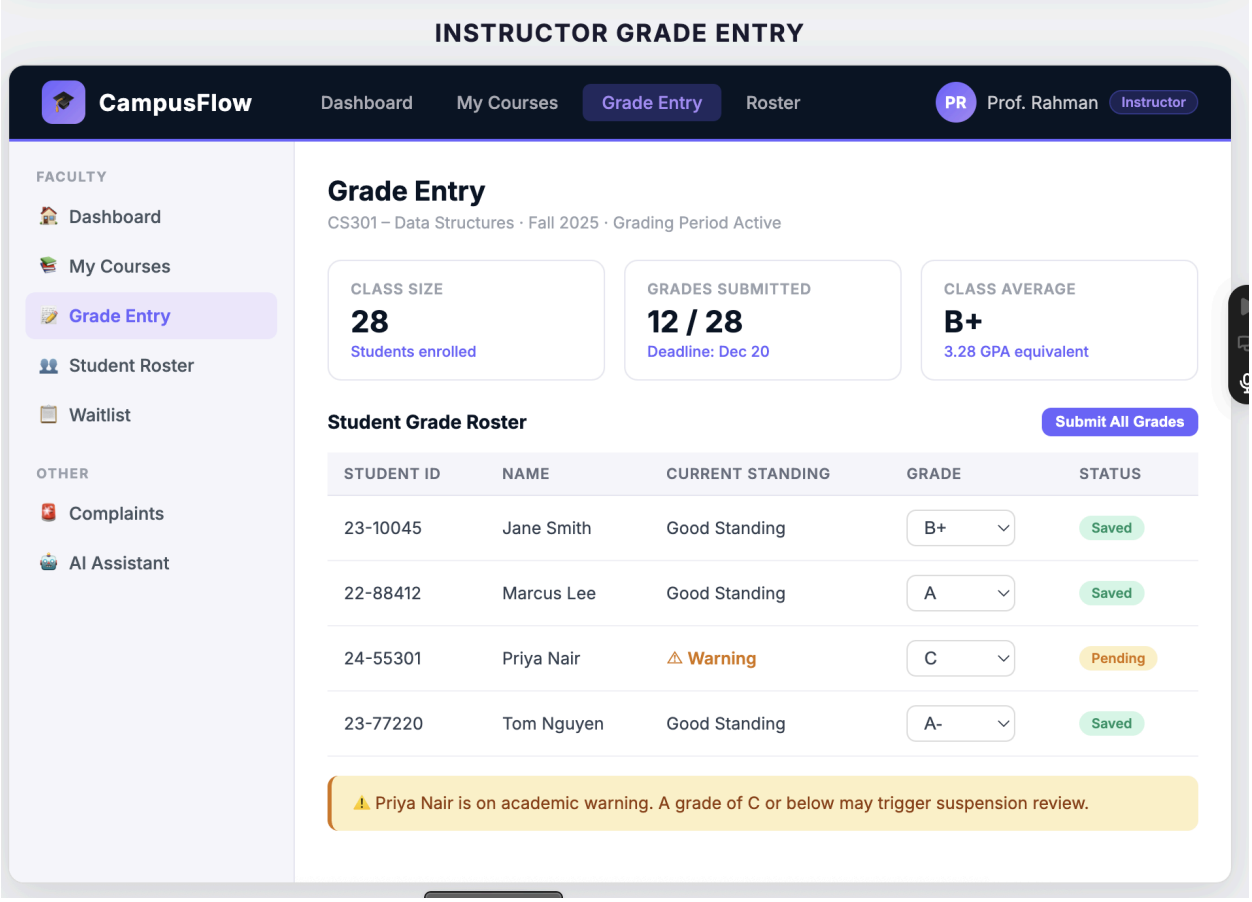
HomePage

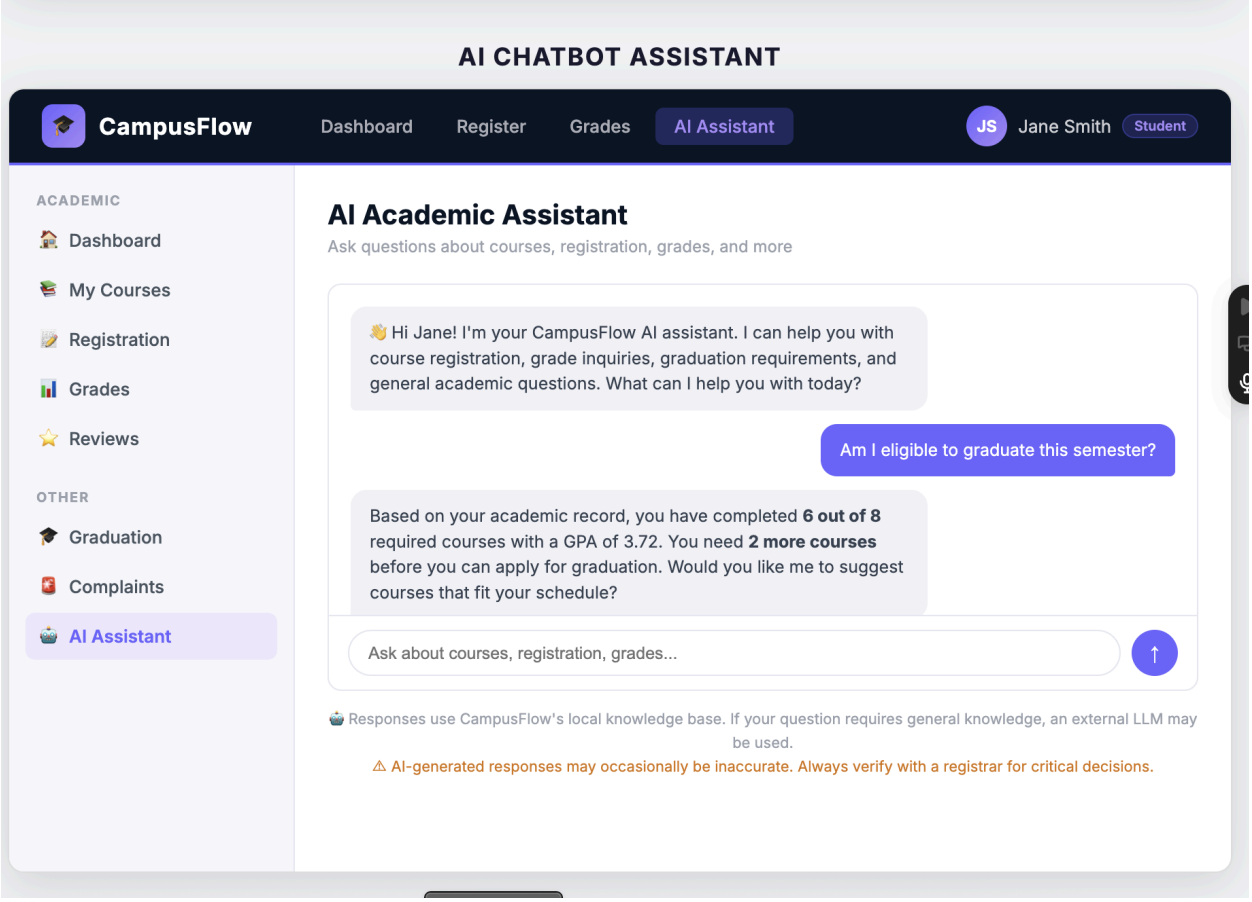


Course Registration



Instructor Page


INSTRUCTOR GRADE ENTRY
CampusFlow
Dashboard
My Courses
Grade Entry
Roster
PR
Prof. Rahman
Instructor
FACULTY
Dashboard
My Courses
Grade Entry
Student Roster
Waitlist
OTHER
Complaints
AI Assistant
Grade Entry
CS301 – Data Structures · Fall 2025 · Grading Period Active
CLASS SIZE
28
Students enrolled
GRADES SUBMITTED
12 / 28
Deadline: Dec 20
CLASS AVERAGE
B+
3.28 GPA equivalent
Student Grade Roster
Submit All Grades
STUDENT ID
NAME
CURRENT STANDING
GRADE
STATUS
23-10045
Jane Smith
Good Standing
B+
Saved
22-88412
Marcus Lee
Good Standing
A
Saved
24-55301
Priya Nair
Warning
C
Pending
23-77220
Tom Nguyen
Good Standing
A-
Saved
Priya Nair is on academic warning. A grade of C or below may trigger suspension review.

AI CHATBOT

AI CHATBOT ASSISTANT